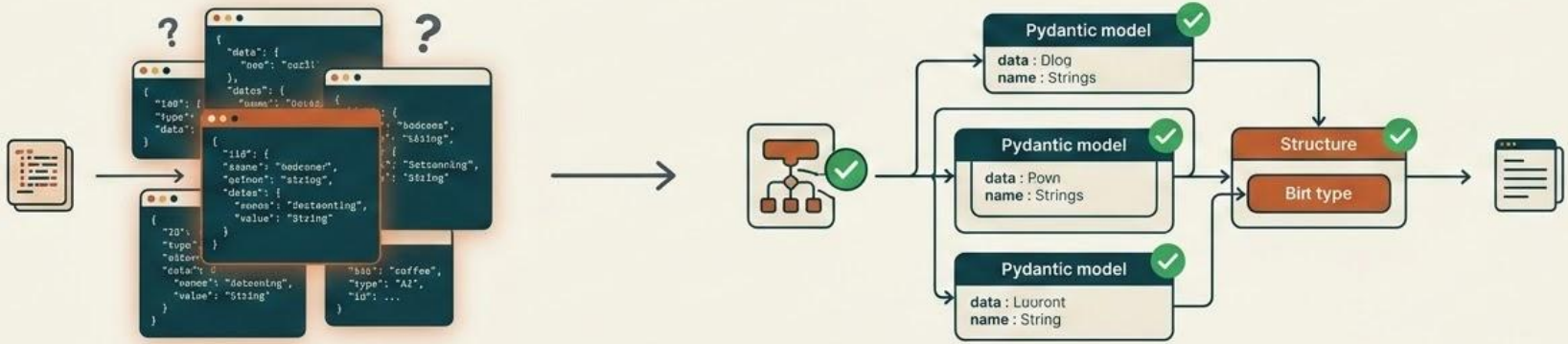
Corporate logo

Pydantic in Modern AI: Building Reliable Agentic Systems

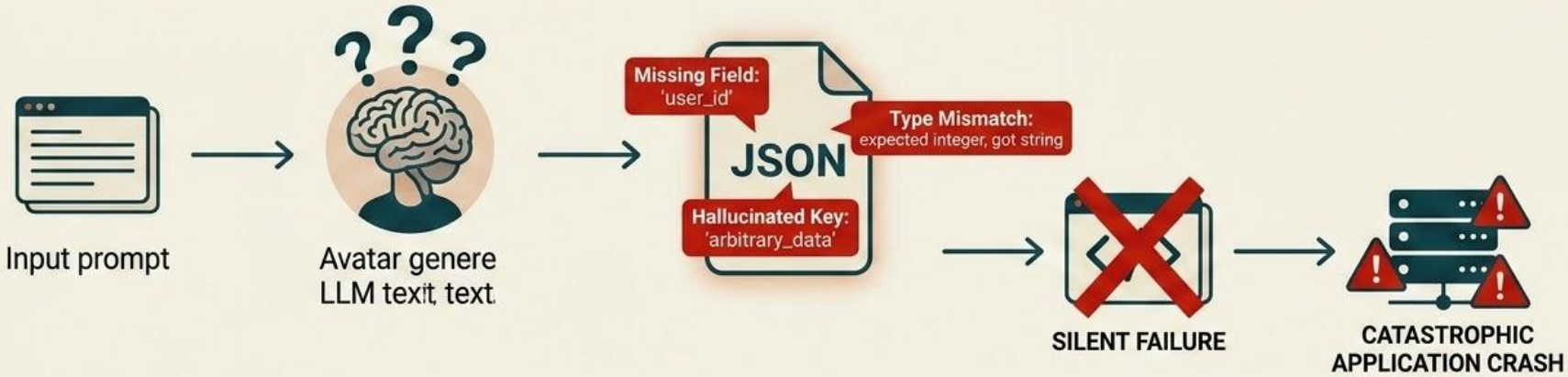
Transitioning from raw JSON schemas to type-safe Python workflows for production-ready LLMs.



Presenter Name | Company Name | Date

The LLM Output Problem

LLMs are probabilistic text predictors, natively generating free-form, unstructured text.



Even in “JSON mode”, LLMs hallucinate keys, omit required fields, and return incorrect data types. Standard JSON parsers fail silently, causing catastrophic crashes in critical downstream application code.

Enter Pydantic: Core Concepts

Defining schemas to guarantee structured, type-safe data flow.

"BaseModel" as the foundation

Using standard Python types (e.g., str, int, Optional) to define structures.

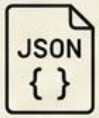
Strict Runtime Validation

Instantly catches incorrect LLM data before it breaks the application.

Native Python Integration

Offers excellent IDE support and static type checking.

The Shift: JSON Schema vs. Pydantic



JSON Schema	Pydantic
• Language-Agnostic • Highly Verbose • Hard to Maintain • Lacks Native Python Runtime Validation	• Python-Native • Clean Syntax • Automatic Runtime Validation • Seamlessly Integrates with AI Frameworks (e.g., LangChain, FastAPI)

Crucial Role in Agentic Systems

Strict Tool Calling

Prevents agents from passing bad arguments to APIs.

Enforcing Structured Outputs

Forces LLMs to reply in predictable formats.

Auto-Correction Loops

Feeding Pydantic validation errors back to the LLM so it can self-correct its mistakes.

Advanced Custom Validations

Beyond Basic Types: How Pydantic leverages advanced, custom validators for robust data integrity.

@field_validator

CLEANS SINGLE FIELDS.
For example, fixing messy capitalization from an LLM output.

@model_validator

CHECKS LOGICAL RULES.
For example, ensuring a Price > 0 if the Status is 'Available'.

Pros & Cons of Pydantic in AI

Pros

- Guarantees strict type safety
- prevents silent crashes
- is the industry standard for AI frameworks

Cons

- It is Python-specific (unlike JSON schema which works everywhere)
- requires learning custom decorator syntax
- introduces a slight performance overhead

Conclusion

- ***Pydantic*** *bridges the gap between unpredictable AI text and strict software engineering.*

Questions?

Open Forum & Discussion